CS6650 | HOMEWORK 1b

ROCKING AWS / EC2 DEPLOYMENT, PERFORMANCE AND CLOUDWATCH

Student	Date	Environment
Nikhil Shivakumar	September 21, 2026	AWS Academy Learner Lab

PURPOSE AND APPROACH

I deployed the Go-Gin album service from Homework 1a on a remotely hosted AWS EC2 virtual machine. I then measured client-observed request latency using a Python script running on my Mac and reviewed the EC2 CPUUtilization metric in CloudWatch.

PART I | AWS DEPLOYMENT

Configuration	Observed or configured value
Instance name	CS6650-HW1b
Region	N. Virginia (us-east-1)
Instance type	t2.micro
Operating system	Amazon Linux 2023 (selected during launch)
Public IPv4 at test time	18.205.161.235
App address	http://18.205.161.235:8080/albums
AWS instance state	Running

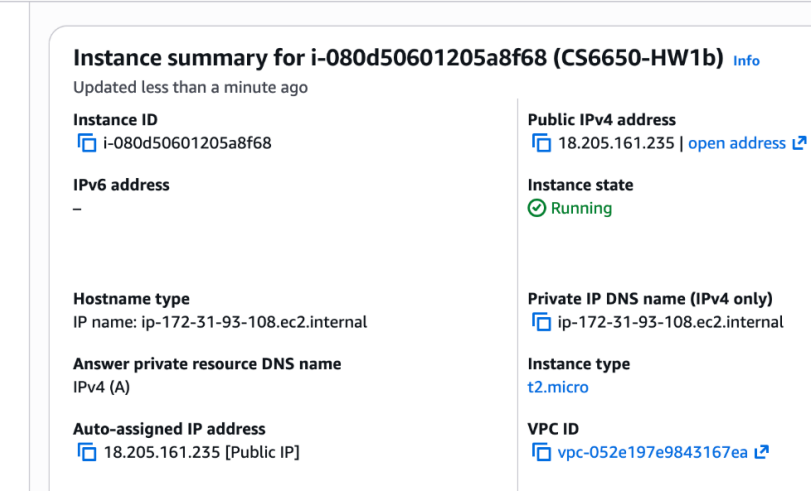


Figure 1. EC2 instance summary: the instance is running with the public address used in the tests.

PART I | DEPLOYMENT EVIDENCE

The server was compiled on the Mac for Linux (amd64), copied to EC2, and started as an executable. The Gin service listened on port 8080; the security group permitted access from the test client.

Server-side log

```
[GIN] 2026/09/21 - 05:32:23 | 200 |  39.295µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:23 | 200 |   37.64µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:23 | 200 |  40.755µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:23 | 200 |  38.302µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:23 | 200 |  43.593µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:23 | 200 |  43.264µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  42.616µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  49.083µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  40.379µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  36.219µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  37.823µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  38.293µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  39.066µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  40.198µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  36.825µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  49.273µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  40.287µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  34.345µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  38.186µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  36.153µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |   35.82µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  49.646µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:24 | 200 |  37.018µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |  38.295µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |  40.833µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |  40.973µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |  50.174µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |  44.517µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |  44.487µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |  46.515µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |   38.78µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |   37.4µs  |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |  40.963µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |   38.94µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |  39.936µs |   73.218.239.11 | GET      "/albu
[GIN] 2026/09/21 - 05:32:25 | 200 |  38.848µs |   73.218.239.11 | GET      "/albu
```

Figure 2. Gin logs on EC2 show repeated GET /albums requests returning HTTP 200 during the load test.

Remote HTTP check

```
Last login: Mon Sep 21 01:21:40 on ttys003
/Users/nikhil/.zshrc:4: command not found: x
nikhil@Nikhils-MacBook-Air ~ % curl -v http://18.205.161.235:8080/albums
*   Trying 18.205.161.235:8080...
* Connected to 18.205.161.235 (18.205.161.235) port 8080
> GET /albums HTTP/1.1
> Host: 18.205.161.235:8080
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Content-Type: application/json; charset=utf-8
< Date: Mon, 21 Sep 2026 05:26:54 GMT
< Content-Length: 382
<
[
    {
        "id": "1",
        "title": "Blue Train",
        "artist": "John Coltrane",
        "price": 56.99
    },
    {
        "id": "2",
        "title": "Jeru",
        "artist": "Gerry Mulligan",
        "price": 17.99
```

Figure 3. curl from the Mac reaches the EC2 public IP and receives HTTP/1.1 200 OK plus the expected three-album JSON payload.

PART II | PERFORMANCE TESTING

I ran a Python requests loop on my Mac, sending valid GET requests sequentially with no intentional pause. Each measured response time covers the client-side round trip to the EC2 /albums endpoint; it is not solely the Gin handler processing time.

Measure	60-second run	300-second run
Total requests	1,047	5,208
Successful requests	1,047	5,208
Failed requests	0	0
Average latency	57.32 ms	57.61 ms
Median (p50)	54.88 ms	54.92 ms
p95	66.70 ms	65.35 ms
p99	76.99 ms	121.25 ms
Maximum	196.07 ms	324.80 ms

Recorded terminal output

```
Matplotlib is building the font cache; this may take a moment.
Starting AWS load test for 60 seconds...

----- PERFORMANCE RESULTS -----
Total requests: 1047
Successful requests: 1047
Failed requests: 0
Average response time: 57.32 ms
Median response time: 54.88 ms
95th percentile: 66.70 ms
```

Figure 4. One-minute experiment: 1,047 completed requests; no recorded failed requests.

```
----- PERFORMANCE RESULTS -----
Total requests: 5208
Successful requests: 5208
Failed requests: 0
Average response time: 57.61 ms
Median response time: 54.92 ms
95th percentile: 65.35 ms
99th percentile: 121.25 ms
```

Figure 5. Five-minute experiment: 5,208 completed requests; no recorded failed requests.

The 300-second experiment used DURATION = 300; the original script had a hard-coded startup message saying “60 seconds.” The displayed 5,208 requests and the 0–300-second plot are from the longer run.

PART II | RESPONSE-TIME VISUALIZATIONS

Both plots below are from the 300-second experiment, rather than the 60-second run. They use the actual measurements captured by the local client.

Distribution of response times

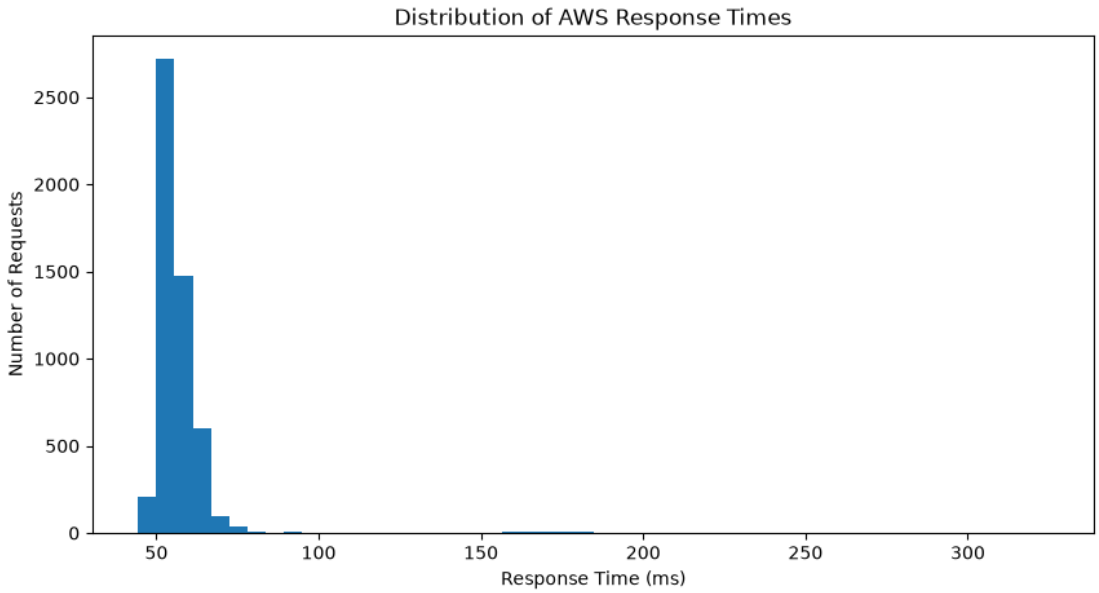


Figure 6. Most requests cluster near 50–65 ms; a small number extend into a long right tail.

Latency over the 5-minute test

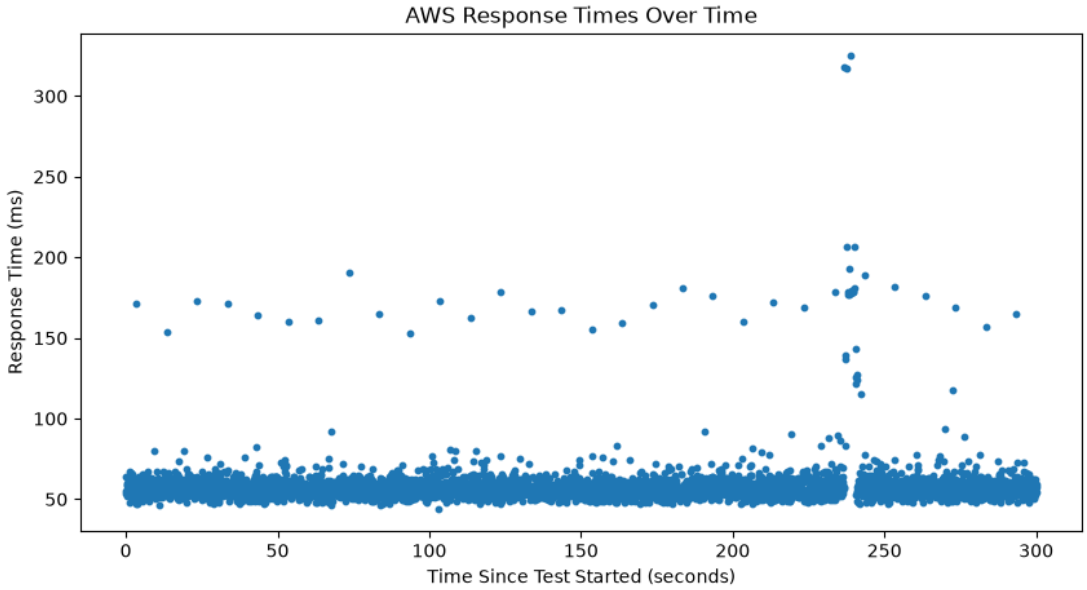


Figure 7. The baseline remains near 50–70 ms, with isolated slow requests and a cluster of larger spikes around 240 seconds.

PART II | QUESTIONS AND INTERPRETATION

1. Distribution shape: is there a long tail, and what proportion is “slow”?

Yes. The 300-second histogram is concentrated around roughly 50–65 ms, while some requests take substantially longer (maximum 324.80 ms). If “slow” is defined as above the measured p95 cutoff of 65.35 ms, approximately 5% fall in that upper tail. A precise percentage above any other cutoff cannot be recovered from the displayed summary alone.

2. Consistency: were there patterns or spikes?

The 300-second scatter plot shows a mostly stable baseline, with occasional spikes throughout the test and a more noticeable set around 240 seconds. There is no sustained upward trend, but individual requests were not equally fast. The assignment question mentions a 30-second window; my actual experiments ran for 60 and 300 seconds.

3. Median versus p95: why does the gap matter?

For the 300-second run, the median was 54.92 ms and p95 was 65.35 ms, a difference of 10.43 ms. The median describes a typical request, while p95 captures the response-time threshold for 95% of requests. The p99 of 121.25 ms highlights a much slower upper tail that the average alone would hide.

4. How might this EC2 setup contribute to variability?

The service ran as a single Go executable on a small t2.micro virtual machine. Shared virtualized resources, instance CPU limits, OS scheduling, and network conditions could contribute to variability, but these results do not isolate their individual effects. This deployment did not demonstrate use of a container.

5. What if there were 100 concurrent users?

This test sends one request after another, so it does not measure 100 simultaneous clients. With real concurrency, the service could experience contention, queuing, or resource limits, increasing latency under some conditions. A separate concurrent load test is needed to measure rather than assume the outcome.

6. Were slow requests caused by the network or server?

The Python measurements include the Mac-to-AWS round trip, while Gin logs show server-side handler durations that are much shorter for the displayed requests. This suggests that network and client-side overhead contribute materially to the end-to-end times, but the screenshots do not identify the cause of each spike. To investigate, compare a test made inside EC2 with one from the Mac, collect per-request server timings, and measure connection and transfer timing separately.

Interpretation: Across both experiments there were no recorded failed requests. The median changed little between the runs, while the longer run exposed a higher p99 and maximum latency, demonstrating why longer samples and tail percentiles are useful.

PART III | CLOUDWATCH MONITORING

CloudWatch was opened in the same AWS region as the EC2 instance. I selected Metrics → EC2 → Per-Instance Metrics → CPUUtilization, using five-minute average periods.

CPUUtilization screenshot

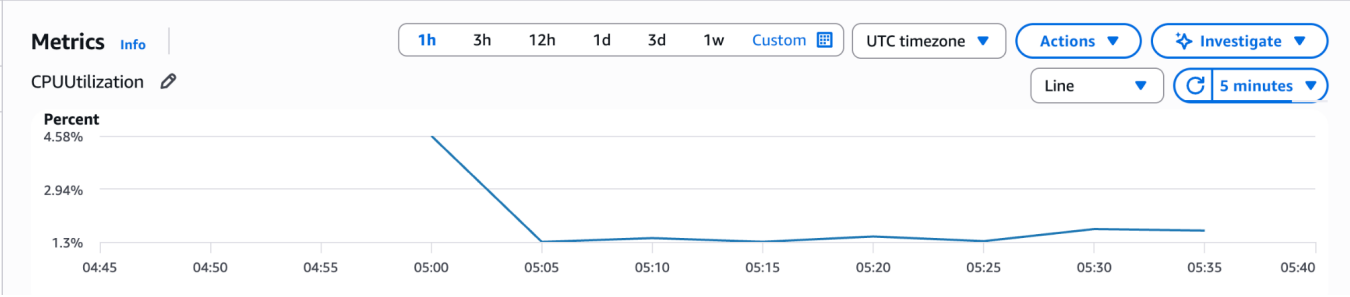


Figure 8. CloudWatch CPUUtilization over one hour. The line shows generally low CPU use with a small increase near 05:30–05:35 UTC.

What CloudWatch can and cannot tell us

CloudWatch shows whether the EC2 instance is reporting CPU usage and how that usage changes over time. However, CPU utilization alone cannot show individual HTTP response times, errors, or the reasons some users experience slower requests. The Python measurements in Part II provide the end-to-end request latency, including its p50, p95, and p99 distribution.

Observed activity and limitation

The graph records approximately 1.3–1.7% CPU utilization for most of the later displayed interval. Because the metric is averaged over five minutes and no precise test-start annotation is shown, the graph supports only a cautious description of modest activity; it does not establish that the load test caused a particular CPU change.

CONCLUSION

The /albums service was reachable from the Mac at the EC2 public address and returned the expected JSON with HTTP 200. In 60- and 300-second sequential tests it recorded 1,047 and 5,208 successful requests, respectively, without recorded request failures. The plots show a stable central latency with occasional slow requests; CloudWatch adds instance-level visibility but does not replace application latency measurements.

Submission contents

Place this report, the Go project files, load_test.py, histogram.png, scatter_plot.png, and relevant screenshot images inside hw01b/. Push the folder to the private course GitHub repository, zip hw01b/ from the Mac, upload the ZIP to Canvas, and paste the repository URL in the same submission. Verify separately that the Piazza reading Note was submitted as required.